

Introduction to Dynamic Programming

The essence of dynamic programming is to avoid repeated calculation. Often, dynamic programming problems are naturally solvable by recursion. In such cases, it's easiest to write the recursive solution, then save repeated states in a lookup table. This process is known as top-down dynamic programming with memoization. That's read "memoization" (like we are writing in a memo pad) not memorization.

One of the most basic, classic examples of this process is the fibonacci sequence. Its recursive formulation is $f(n) = f(n-1) + f(n-2)$ where $n \geq 2$ and $f(0)=0$ and $f(1)=1$. In C++, this would be expressed as:

```
int f(int n) {  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
    return f(n - 1) + f(n - 2);  
}
```

The runtime of this recursive function is exponential - approximately $O(2^n)$ since one function call ($f(n)$) results in 2 similarly sized function calls ($f(n-1)$ and $f(n-2)$).

Speeding up Fibonacci with Dynamic Programming (Memoization)

Our recursive function currently solves fibonacci in exponential time. This means that we can only handle small input values before the problem becomes too difficult. For instance, $f(29)$ results in over 1 million function calls!

To increase the speed, we recognize that the number of subproblems is only $O(n)$. That is, in order to calculate $f(n)$ we only need to know $f(n-1), f(n-2), \dots, f(0)$. Therefore, instead of recalculating these subproblems, we solve them once and then save the result in a lookup table. Subsequent calls will use this lookup table and immediately return a result, thus eliminating exponential work!

Each recursive call will check against a lookup table to see if the value has been calculated. This is done in $O(1)$ time. If we have previously calculated it, return the result, otherwise, we calculate the function normally. The overall runtime is $O(n)$. This is an enormous improvement over our previous exponential time algorithm!